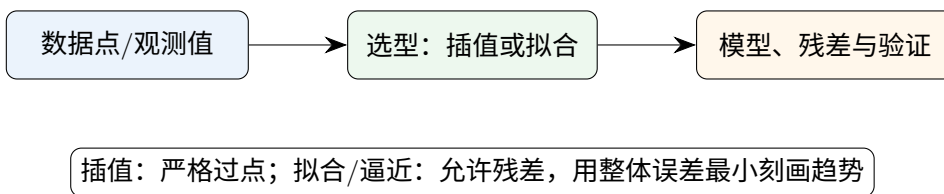


常用插值算法、曲线拟合与最小二乘建模方法

拉格朗日、牛顿、埃尔米特、三次样条、二维插值与 Python 实现



适合：数值分析复习、建模论文方法说明、Python 实验代码模板

生成日期：2026 年 7 月 9 日

目录

1 总览：什么时候用插值，什么时候用拟合？	2
1.1 常用方法速查表	2
1.2 建模通用流程	3
2 曲线拟合与最小二乘法	4
2.1 线性最小二乘模型	4
2.2 多项式拟合：最常见的线性最小二乘	4
2.3 最小二乘法优化：从线性到非线性	5
2.4 加权最小二乘与正则化	6
2.5 拟合结果怎么写成建模论文语言？	6
3 曲线拟合与函数逼近	7
3.1 函数逼近的基本思想	7
3.2 插值、拟合、逼近的关系	7
3.3 正交基与最佳平方逼近	7
4 拉格朗日插值法	9
4.1 原理与公式	9
4.2 误差公式与使用条件	9
4.3 步骤	9
4.4 Python 实现	9
4.5 小例题	10
5 牛顿插值法	11
5.1 差商与公式	11
5.2 优势	11
5.3 差商表代码	11
5.4 小例题	12
6 埃尔米特插值法（Hermite 插值）	13
6.1 名称说明	13
6.2 基本思想	13
6.3 Python 实现	13
6.4 考试写法示例	14
7 三次样条插值	15
7.1 为什么需要样条？	15

7.2	常见边界条件	15
7.3	三次样条方程思想	15
7.4	Python 实现	16
7.5	建模建议	16
8	二维插值	17
8.1	二维插值的两类数据	17
8.2	双线性插值公式	17
8.3	规则网格代码	18
8.4	散点数据代码	18
8.5	二维插值选型建议	19
9	方法对比与选型指南	20
9.1	按问题目标选方法	20
9.2	按考试题型选方法	20
9.3	按建模论文写法选方法	20
10	完整 Python 代码模板	21
11	常见错误与改进策略	23
11.1	错误一：把含噪声数据强行插值	23
11.2	错误二：多项式阶数越高越好	23
11.3	错误三：直接求逆解最小二乘	23
11.4	错误四：忽略外推风险	23
11.5	错误五：没有做残差诊断	23
12	复习结论	24
13	参考资料	24

1 总览：什么时候用插值，什么时候用拟合？

核心区分

插值要求构造函数 $s(x)$ 使 $s(x_i) = y_i$ ，适合数据较准确、节点数不太多、希望在节点之间补值的情形。曲线拟合/函数逼近不要求过每个点，而是让残差总体尽量小，适合观测含噪声、希望找趋势、要预测或解释参数的情形。

常见建模问题可以先按数据性质分为三类：

- 1) 无噪声或误差很小的数据表：如函数表、物理标定表、工程查表。优先考虑线性插值、三次样条、Hermite 插值或二维插值。
- 2) 含噪声观测数据：如实验测量、统计样本、传感器数据。优先考虑最小二乘、多项式拟合、非线性最小二乘、正则化拟合。
- 3) 想用简单函数逼近复杂函数：如考试中的最佳平方逼近、建模中的低维替代模型。优先考虑正交基展开、Chebyshev 多项式、分段多项式或样条逼近。

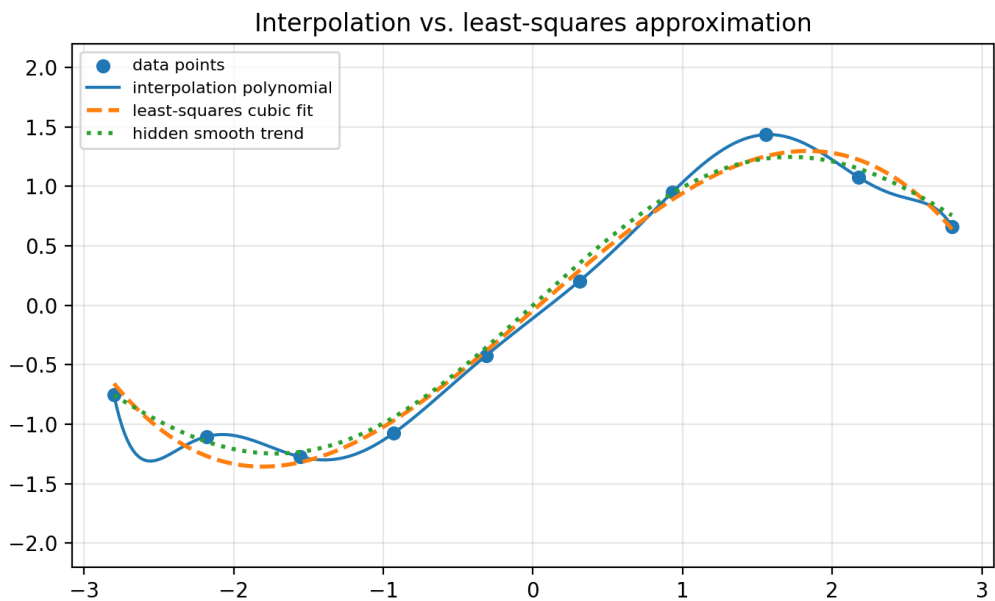


图 1：插值和最小二乘拟合的直观区别：插值多项式过所有点，但高次多项式可能振荡；最小二乘三次拟合不过每个点，却更能体现整体趋势。

1.1 常用方法速查表

方法	目标	适用条件	注意事项
拉格朗日插值	构造唯一 n 次以内多项式过 $n + 1$ 点	节点互异，数据较准确	节点多时不宜直接展开，优先用重心形式；等距高次会有 Runge 振荡

牛顿插值	用差商递推构造插值多项式	节点互异，可逐步加入新点	适合手算和递增节点；数值稳定性仍受高次多项式影响
埃尔米特插值	同时匹配函数值和导数值	已知节点处斜率/导数	信息更强，曲线更光滑；导数估计不准会放大误差
三次样条插值	分段三次，通常要求 C^2 光滑	一维有序节点，想要平滑补值	需选边界条件：自然、夹持、not-a-knot、周期
二维插值	在平面/网格上补值	规则网格或散点数据	规则网格用 <code>RegularGridInterpolator</code> ；散点用 <code>triangulation/-griddata</code>
线性最小二乘	最小化 $\ A\beta - y\ _2^2$	模型对参数线性，样本通常多于参数	优先用 QR/SVD 或库函数，不建议直接求逆
多项式拟合	用低阶多项式逼近趋势	数据含噪，趋势平滑	阶数过高会过拟合；建议中心化/标准化
非线性最小二乘	最小化残差平方和 $\sum r_i(\theta)^2$	模型对参数非线性	依赖初值；可加边界、鲁棒损失、参数缩放
函数逼近	在某个范数下逼近函数	已知函数或可采样，关注整体误差	选择基函数和误差度量最关键

1.2 建模通用流程

一套通用建模步骤

1) 整理数据：确认自变量维度、单位、是否重复节点、是否有缺失值、是否按 x 排序。

2) 判断误差性质：数据是否必须全部通过？若观测含噪声，通常不宜插值，而应用拟合。

3) 选择函数族：多项式、指数、幂函数、样条、分段函数、二维曲面或物理模型。

4) 建立目标函数：插值是满足约束 $s(x_i) = y_i$ ；拟合是最小化残差平方和或加权残差。

5) 求解参数/插值函数：手算可用公式和线性方程组；编程用 NumPy/SciPy。

6) 验证与诊断：画残差图、计算 RMSE/ R^2 、留出验证、检查外推风险。

7) 写结论：说明方法、条件、参数、误差、适用范围和不能外推的区间。

2 曲线拟合与最小二乘法

2.1 线性最小二乘模型

设有观测数据 (x_i, y_i) ，选取基函数 $\phi_0(x), \phi_1(x), \dots, \phi_m(x)$ ，用

$$\hat{y} = f(x; \beta) = \sum_{j=0}^m \beta_j \phi_j(x)$$

拟合数据。令设计矩阵

$$A = \begin{bmatrix} \phi_0(x_1) & \phi_1(x_1) & \cdots & \phi_m(x_1) \\ \phi_0(x_2) & \phi_1(x_2) & \cdots & \phi_m(x_2) \\ \vdots & \vdots & & \vdots \\ \phi_0(x_n) & \phi_1(x_n) & \cdots & \phi_m(x_n) \end{bmatrix}, \quad \mathbf{y} = (y_1, \dots, y_n)^T.$$

线性最小二乘问题为

$$\min_{\beta} \|A\beta - \mathbf{y}\|_2^2.$$

当 A 列满秩时，正规方程为

$$A^T A \beta = A^T \mathbf{y}.$$

但实际计算不建议显式求 $(A^T A)^{-1}$ ，因为会放大病态性。数值计算通常使用 QR 分解、SVD 或库函数 `numpy.linalg.lstsq`。NumPy 文档也将该接口表述为求 $ax = b$ 的最小二乘解，即最小化 $\|b - ax\|_2$ 。

使用条件

- 模型对未知参数 β_j 是线性的，即参数只以一次形式出现。
- 样本数 n 通常大于参数个数 $m + 1$ ，否则可能解不唯一。
- 残差最好没有明显系统结构；若方差不等，可用加权最小二乘。

2.2 多项式拟合：最常见的线性最小二乘

若取基函数 $1, x, x^2, \dots, x^m$ ，则

$$f(x) = \beta_0 + \beta_1 x + \cdots + \beta_m x^m.$$

这就是多项式拟合。阶数越高，训练误差通常越小，但不代表预测更好。高阶多项式会带来振荡、病态矩阵和过拟合。实际建模一般从低阶开始，通过残差图、验证误差或信息准则选阶。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.array([0, 1, 2, 3, 4, 5], dtype=float)
5 y = np.array([1.0, 2.1, 2.8, 4.2, 5.1, 6.0])
6
7 # Fit y = beta0 + beta1*x + beta2*x^2
8 A = np.vstack([np.ones_like(x), x, x**2]).T
9 beta, residuals, rank, s = np.linalg.lstsq(A, y, rcond=None)
```

```

10 print(beta)
11
12 xx = np.linspace(x.min(), x.max(), 200)
13 yy = beta[0] + beta[1]*xx + beta[2]*xx**2
14 plt.scatter(x, y)
15 plt.plot(xx, yy)
16 plt.show()

```

2.3 最小二乘法优化：从线性到非线性

很多模型对参数不是线性的，如指数衰减、Logistic 曲线、Michaelis-Menten 模型：

$$y = ae^{bx} + c, \quad y = \frac{L}{1 + e^{-k(x-x_0)}}, \quad y = \frac{V_{max}x}{K_m + x}.$$

这时通常写成残差向量

$$r_i(\theta) = f(x_i; \theta) - y_i, \quad \min_{\theta} \frac{1}{2} \sum_{i=1}^n r_i(\theta)^2.$$

非线性最小二乘通常迭代求解，常见算法包括 Gauss-Newton、Levenberg-Marquardt、信赖域反射法等。SciPy 的 `curve_fit` 主要用于曲线拟合参数估计；`least_squares` 更通用，可以直接给残差函数、参数边界和鲁棒损失。

非线性最小二乘的三件事

- 1) **初值很重要**：非线性问题可能陷入局部最优或不收敛。
- 2) **参数尺度要合理**：不同参数量纲差太大会影响迭代。
- 3) **要检查残差**：拟合结果看起来通过很多点，不代表模型正确。

```

1 import numpy as np
2 from scipy.optimize import curve_fit, least_squares
3
4 # Nonlinear model: y = a * exp(b*x) + c
5 def model(x, a, b, c):
6     return a * np.exp(b*x) + c
7
8 x = np.linspace(0, 4, 30)
9 rng = np.random.default_rng(0)
10 y = model(x, 2.0, -0.8, 0.5) + rng.normal(0, 0.08, size=x.size)
11
12 # curve_fit: convenient curve fitting interface
13 popt, pcov = curve_fit(model, x, y, p0=[1.0, -1.0, 0.0])
14 print(popt)
15
16 # least_squares: directly define residuals and constraints
17 def residual(theta):
18     a, b, c = theta
19     return model(x, a, b, c) - y
20
21 res = least_squares(

```

```
22     residual,  
23     x0=[1.0, -1.0, 0.0],  
24     bounds=([0.0, -5.0, -np.inf], [10.0, 0.0, np.inf]),  
25     loss='soft_l1'  
26 )  
27 print(res.x)
```

2.4 加权最小二乘与正则化

若第 i 个观测的标准差为 σ_i ，可最小化

$$\sum_{i=1}^n \left(\frac{f(x_i) - y_i}{\sigma_i} \right)^2.$$

这等价于给可靠数据更大权重。若模型参数很多或矩阵病态，可加正则化：

$$\min_{\beta} \|A\beta - \mathbf{y}\|_2^2 + \lambda \|\beta\|_2^2,$$

即岭回归。正则化的作用是牺牲一点拟合精度，换取更稳定的参数和更好的泛化。

2.5 拟合结果怎么写成建模论文语言？

规范表述模板

本文采用最小二乘法拟合变量 x 与响应 y 的关系。设模型为 $f(x; \theta)$ ，以观测值与模型值之间的残差平方和

$$S(\theta) = \sum_{i=1}^n \{y_i - f(x_i; \theta)\}^2$$

作为目标函数。通过数值优化得到参数估计 $\hat{\theta}$ ，并利用残差图、均方根误差 RMSE 和决定系数 R^2 检验拟合效果。若残差随机分布且验证误差较小，说明模型能够较好刻画数据趋势。

3 曲线拟合与函数逼近

3.1 函数逼近的基本思想

曲线拟合是由离散数据出发；函数逼近则常常从一个已知函数 $f(x)$ 出发，用简单函数 $p(x)$ 近似它。常见目标包括：

$$\min_{p \in V} \max_{x \in [a,b]} |f(x) - p(x)| \quad \text{一致逼近,}$$

或

$$\min_{p \in V} \int_a^b [f(x) - p(x)]^2 w(x) \, dx \quad \text{最佳平方逼近.}$$

其中 V 是候选函数空间，比如所有次数不超过 m 的多项式空间。数值分析考试中常见的是最佳平方逼近，建模中常见的是用有限维参数模型压缩复杂函数。

3.2 插值、拟合、逼近的关系

概念	数据来源	数学要求	典型问题
插值	离散节点, 通常认为准确	必须过所有节点	查表补值、三次样条、 二维网格插值
拟合	离散观测, 通常含噪声	残差整体最小	实验数据趋势、回归、 参数估计
函数逼近	已知函数或可大量采样	某种范数误差最小	最佳平方逼近、 Chebyshev 逼近、数值 算法替代模型

3.3 正交基与最佳平方逼近

若 $\{\phi_0, \dots, \phi_m\}$ 在内积

$$\langle f, g \rangle = \int_a^b f(x)g(x)w(x) \, dx$$

下正交，则最佳平方逼近

$$p_m(x) = \sum_{k=0}^m c_k \phi_k(x)$$

的系数为

$$c_k = \frac{\langle f, \phi_k \rangle}{\langle \phi_k, \phi_k \rangle}.$$

这就是“误差与逼近空间正交”的思想。若基函数不是正交的，就要解正规方程。

建模中如何选择基函数？

- 数据在短区间内平滑：低阶多项式或三次样条。
- 有周期性：三角函数基或 Fourier 逼近。
- 有指数增长/衰减：指数函数、对数变换或非线性最小二乘。
- 分段趋势明显：分段线性、分段多项式或样条。
- 高维但样本少：低阶模型、正则化或物理先验比盲目高阶更可靠。

4 拉格朗日插值法

4.1 原理与公式

给定 $n+1$ 个互异节点 $(x_0, y_0), \dots, (x_n, y_n)$, 存在唯一次数不超过 n 的多项式 $P_n(x)$ 满足

$$P_n(x_i) = y_i, \quad i = 0, 1, \dots, n.$$

拉格朗日基函数定义为

$$L_i(x) = \prod_{\substack{0 \leq j \leq n \\ j \neq i}} \frac{x - x_j}{x_i - x_j}.$$

于是插值多项式为

$$P_n(x) = \sum_{i=0}^n y_i L_i(x).$$

由于 $L_i(x_j) = \delta_{ij}$, 代入任一节点 x_j 时只剩 y_j , 因此它一定过所有节点。

4.2 误差公式与使用条件

若 $f \in C^{n+1}[a, b]$, 且 $y_i = f(x_i)$, 则对任意 $x \in [a, b]$, 存在 ξ 介于节点和 x 之间, 使

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i).$$

该公式说明误差由两部分控制: 函数高阶导数大小、节点多项式 $\prod (x - x_i)$ 。等距节点且次数很高时, 端点附近可能出现明显振荡, 这就是 Runge 现象的典型来源。

不要直接展开高次拉格朗日多项式

手算小题可以写 $L_i(x)$, 但编程中不建议直接计算展开系数。实际数值计算更常使用重心拉格朗日形式, 因为它更稳定, 也便于更新函数值。SciPy 的 `BarycentricInterpolator` 就是采用更稳定的重心表示。

4.3 步骤

- 1) 检查节点 x_i 是否互异。
- 2) 写出每个基函数 $L_i(x)$ 。
- 3) 计算 $P_n(x) = \sum y_i L_i(x)$ 。
- 4) 若要求具体点 x^* 的估计值, 代入 $P_n(x^*)$ 。
- 5) 若需要误差估计, 使用误差公式并估计 $\max |f^{(n+1)}|$ 。

4.4 Python 实现

```

1 import numpy as np
2 from scipy.interpolate import BarycentricInterpolator
3
4 def lagrange_eval(x_nodes, y_nodes, x_eval):
5     x_nodes = np.asarray(x_nodes, dtype=float)
6     y_nodes = np.asarray(y_nodes, dtype=float)
7     x_eval = np.asarray(x_eval, dtype=float)
8     out = np.zeros_like(x_eval, dtype=float)
9     n = len(x_nodes)
10    for i in range(n):
11        Li = np.ones_like(x_eval, dtype=float)
12        for j in range(n):
13            if i != j:
14                Li *= (x_eval - x_nodes[j]) / (x_nodes[i] - x_nodes[j])
15        out += y_nodes[i] * Li
16    return out
17
18 x_nodes = np.array([0.0, 1.0, 2.0, 4.0])
19 y_nodes = np.array([1.0, 2.0, 0.0, 3.0])
20 xx = np.linspace(0, 4, 100)
21
22 # Direct formula, suitable for teaching and small n
23 p1 = lagrange_eval(x_nodes, y_nodes, xx)
24
25 # Recommended stable implementation
26 interp = BarycentricInterpolator(x_nodes, y_nodes)
27 p2 = interp(xx)

```

4.5 小例题

已知 $f(0) = 1, f(1) = 2, f(2) = 5$ ，求二次插值多项式。

$$L_0 = \frac{(x-1)(x-2)}{(0-1)(0-2)} = \frac{(x-1)(x-2)}{2},$$

$$L_1 = \frac{x(x-2)}{(1-0)(1-2)} = -x(x-2), \quad L_2 = \frac{x(x-1)}{(2-0)(2-1)} = \frac{x(x-1)}{2}.$$

所以

$$P_2(x) = 1 \cdot L_0 + 2 \cdot L_1 + 5 \cdot L_2 = x^2 + 1.$$

5 牛顿插值法

5.1 差商与公式

牛顿插值使用差商构造多项式：

$$P_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + \cdots + f[x_0, \dots, x_n] \prod_{k=0}^{n-1} (x - x_k).$$

一阶差商为

$$f[x_i, x_j] = \frac{f(x_j) - f(x_i)}{x_j - x_i},$$

高阶差商递推为

$$f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

5.2 优势

牛顿形式和拉格朗日形式表示的是同一个插值多项式，但牛顿法更适合逐步加入节点。若已有 $P_n(x)$ ，增加新节点 x_{n+1} 时，只需多算一个新的差商系数，而不必重写所有基函数。

使用条件

- 节点 x_i 互异。
- 适合节点不断增加、手算差商表、或需要构造递推形式。
- 与拉格朗日一样，高次数插值仍可能振荡。

5.3 差商表代码

```

1  import numpy as np
2
3  def newton_coefficients(x, y):
4      x = np.asarray(x, dtype=float)
5      coef = np.asarray(y, dtype=float).copy()
6      n = len(x)
7      for j in range(1, n):
8          coef[j:n] = (coef[j:n] - coef[j-1:n-1]) / (x[j:n] - x[0:n-j])
9      return coef
10
11 def newton_eval(coef, x_nodes, x_eval):
12     x_eval = np.asarray(x_eval, dtype=float)
13     n = len(coef)
14     p = np.zeros_like(x_eval) + coef[-1]
15     for k in range(n-2, -1, -1):
16         p = coef[k] + (x_eval - x_nodes[k]) * p
17     return p
18
19 x = np.array([0.0, 1.0, 2.0, 4.0])
20 y = np.array([1.0, 2.0, 5.0, 17.0])

```

```
21 coef = newton_coefficients(x, y)
22 print(coef)
23 print(newton_eval(coef, x, np.array([3.0])))
```

5.4 小例题

给定 $(0, 1), (1, 2), (2, 5)$ 。差商表：

$$\begin{aligned} f[x_0] &= 1, & f[x_0, x_1] &= 1, \\ f[x_1, x_2] &= 3, & f[x_0, x_1, x_2] &= \frac{3-1}{2-0} = 1. \end{aligned}$$

所以

$$P_2(x) = 1 + 1(x-0) + 1(x-0)(x-1) = x^2 + 1.$$

这与拉格朗日插值得到的结果相同。

6 埃尔米特插值法 (Hermite 插值)

6.1 名称说明

用户常写“埃米尔特”，标准名称一般写作**埃尔米特插值**或 Hermite interpolation。它不仅要求函数值相同，还要求某些导数值相同。

6.2 基本思想

普通插值只使用 $f(x_i)$ ，而 Hermite 插值还使用 $f'(x_i), f''(x_i)$ 等信息。例如三次 Hermite 插值在区间 $[x_0, x_1]$ 上给定

$$f(x_0), \quad f(x_1), \quad f'(x_0), \quad f'(x_1),$$

构造一个三次多项式 $H_3(x)$ 同时满足

$$H_3(x_0) = f(x_0), \quad H_3(x_1) = f(x_1),$$

$$H_3'(x_0) = f'(x_0), \quad H_3'(x_1) = f'(x_1).$$

令 $t = (x - x_0)/h$, $h = x_1 - x_0$ ，则三次 Hermite 形式为

$$H(x) = h_{00}(t)y_0 + h_{10}(t)hm_0 + h_{01}(t)y_1 + h_{11}(t)hm_1,$$

其中

$$h_{00} = 2t^3 - 3t^2 + 1, \quad h_{10} = t^3 - 2t^2 + t,$$

$$h_{01} = -2t^3 + 3t^2, \quad h_{11} = t^3 - t^2.$$

这里 $m_0 = f'(x_0), m_1 = f'(x_1)$ 。

适用条件

- 已知函数值和导数值，或能较准确估计导数。
- 希望曲线在节点处不仅连续，而且斜率也合理。
- 工程制图、轨迹规划、动画曲线、物理量平滑连接中常用。

导数值不准时的风险

Hermite 插值把导数当作硬约束。如果导数来自噪声数据的差分估计，误差可能导致曲线过冲。此时可考虑 PCHIP、平滑样条或先做拟合再求导。

6.3 Python 实现

```
1 import numpy as np
2 from scipy.interpolate import CubicHermiteSpline
3
4 def cubic_hermite_segment(x0, x1, y0, y1, m0, m1, x):
5     h = x1 - x0
6     t = (x - x0) / h
7     h00 = 2*t**3 - 3*t**2 + 1
```

```
8     h10 = t**3 - 2*t**2 + t
9     h01 = -2*t**3 + 3*t**2
10    h11 = t**3 - t**2
11    return h00*y0 + h10*h*m0 + h01*y1 + h11*h*m1
12
13    x = np.array([0.0, 1.0, 2.0, 3.0])
14    y = np.sin(x)
15    dydx = np.cos(x)
16    xx = np.linspace(0, 3, 200)
17
18    spline = CubicHermiteSpline(x, y, dydx)
19    yy = spline(xx)
```

6.4 考试写法示例

若题目给出 $f(0) = 1, f(1) = 2, f'(0) = 0, f'(1) = 3$, 设 $x_0 = 0, x_1 = 1, h = 1, t = x$ 。代入三次 Hermite 基函数:

$$\begin{aligned} H(x) &= (2x^3 - 3x^2 + 1) \cdot 1 + (x^3 - 2x^2 + x) \cdot 0 \\ &\quad + (-2x^3 + 3x^2) \cdot 2 + (x^3 - x^2) \cdot 3. \end{aligned}$$

整理得

$$H(x) = x^3 + x^2 + 1.$$

7 三次样条插值

7.1 为什么需要样条？

高次全局多项式插值容易振荡，而且任意一个节点变化都会影响整个区间。样条插值采用**分段低次多项式**：在每个小区间 $[x_i, x_{i+1}]$ 上用三次多项式，但在节点处拼接得足够光滑。

三次样条 $S(x)$ 通常满足：

- 1) 在每个区间 $[x_i, x_{i+1}]$ 上， $S(x)$ 是三次多项式；
- 2) 插值条件： $S(x_i) = y_i$ ；
- 3) 光滑条件： $S'(x)$ 与 $S''(x)$ 在内节点连续；
- 4) 加上两个边界条件后解唯一。

SciPy 的 CubicSpline 文档将其描述为二阶连续可微的分段三次插值器，即 C^2 光滑的 piecewise cubic interpolator。

7.2 常见边界条件

边界条件	数学含义	适用情形
自然边界 natural	$S''(x_0) = S''(x_n) = 0$	两端弯曲程度未知, 想让端点更“自然”
夹持边界 clamped	指定 $S'(x_0), S'(x_n)$	已知端点斜率, 如速度、增长率
not-a-knot	前两个/后两个区间在更高阶意义上合并	SciPy 默认常用, 端点无额外信息时可用
周期 periodic	函数值和导数在两端周期衔接	周期数据, 如角度、季节周期

7.3 三次样条方程思想

设 $M_i = S''(x_i)$, $h_i = x_{i+1} - x_i$ 。对内节点可建立三对角方程：

$$h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_iM_{i+1} = 6 \left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right).$$

再加上两个边界条件即可解出所有 M_i ，进而得到每段三次多项式。

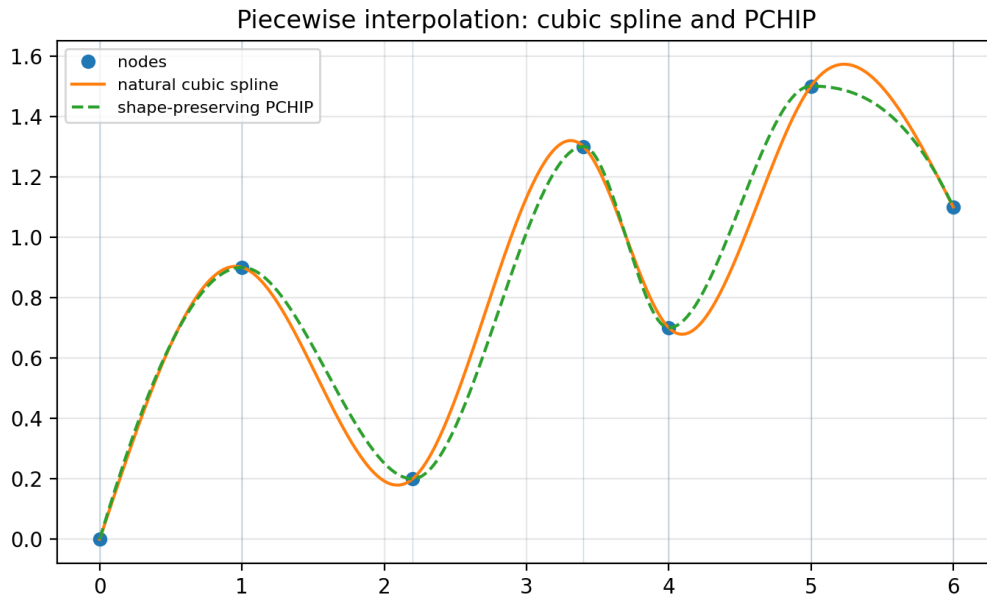


图 2: 三次样条与保形 PCHIP 的比较: 三次样条更平滑; PCHIP 更强调保持单调性和形状, 减少过冲。

7.4 Python 实现

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.interpolate import CubicSpline, PchipInterpolator
4
5 x = np.array([0, 1, 2.2, 3.4, 4.0, 5.0, 6.0], dtype=float)
6 y = np.array([0, 0.9, 0.2, 1.3, 0.7, 1.5, 1.1], dtype=float)
7 xx = np.linspace(x.min(), x.max(), 400)
8
9 cs_natural = CubicSpline(x, y, bc_type='natural')
10 cs_clamped = CubicSpline(x, y, bc_type=((1, 0.6), (1, -0.2)))
11 pchip = PchipInterpolator(x, y)
12
13 plt.plot(x, y, 'o')
14 plt.plot(xx, cs_natural(xx), label='natural cubic spline')
15 plt.plot(xx, cs_clamped(xx), label='clamped cubic spline')
16 plt.plot(xx, pchip(xx), label='PCHIP')
17 plt.legend()
18 plt.show()
```

7.5 建模建议

- 若数据真实无噪声, 只是想补值, 三次样条很常用。
- 若数据含噪声, 插值样条会把噪声也穿过去, 建议使用平滑样条或最小二乘拟合。
- 若数据有单调性、不能出现过冲, 可考虑 PCHIP 而不是普通三次样条。
- 外推通常不可靠, 尤其是样条端点外的延伸。

8 二维插值

8.1 二维插值的两类数据

二维插值一般给定点 (x_i, y_i) 上的函数值 z_i ，希望估计新点 (x, y) 处的 z 。关键是先判断数据是规则网格还是散点数据。

数据类型	特征	推荐方法
规则网格	x 坐标和 y 坐标组成矩形网格， z 是二维数组	双线性/双三次插值，SciPy <code>RegularGridInterpolator</code>
散点数据	点随机分布，不形成完整网格	最近邻、线性三角剖分、cubic grid-data、RBF、克里金等
图像数据	像素网格，通常等距	最近邻、双线性、双三次、Lanczos 等图像插值

SciPy 文档说明，`RegularGridInterpolator` 适用于直角网格/rectilinear grid 上的多维数据；`griddata` 是多维非结构散点数据的便捷插值函数，教程中特别提醒散点方法基于三角剖分，若数据已经在完整网格上则应使用 `RegularGridInterpolator`。

8.2 双线性插值公式

在矩形网格单元 $(x_0, y_0), (x_1, y_0), (x_0, y_1), (x_1, y_1)$ 内，设

$$t = \frac{x - x_0}{x_1 - x_0}, \quad u = \frac{y - y_0}{y_1 - y_0}.$$

双线性插值为

$$z(x, y) = (1 - t)(1 - u)z_{00} + t(1 - u)z_{10} + (1 - t)uz_{01} + tuz_{11}.$$

它先沿 x 插值，再沿 y 插值，或者反过来结果相同。

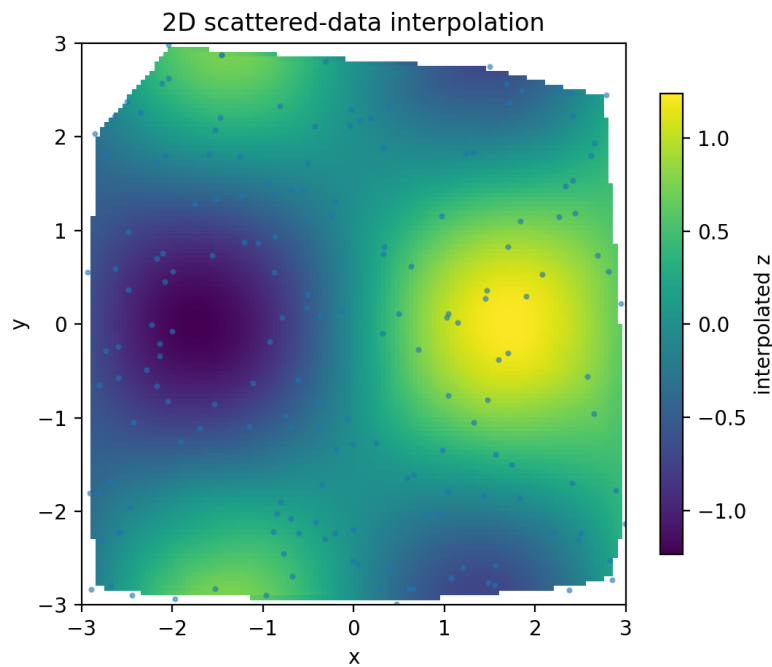


图 3: 二维散点插值示意: 黑点为采样点, 背景为由散点估计出的连续曲面。

8.3 规则网格代码

```
1 import numpy as np
2 from scipy.interpolate import RegularGridInterpolator
3
4 # Structured grid
5 x = np.linspace(-2, 2, 21)
6 y = np.linspace(-3, 3, 31)
7 X, Y = np.meshgrid(x, y, indexing='ij')
8 Z = np.sin(X) * np.cos(Y)
9
10 interp = RegularGridInterpolator(
11     (x, y), Z,
12     method='linear',
13     bounds_error=False,
14     fill_value=np.nan
15 )
16
17 query_points = np.array([
18     [0.1, 0.2],
19     [1.3, -2.1],
20     [2.5, 0.0]
21 ])
22 print(interp(query_points))
```

8.4 散点数据代码

```
1 import numpy as np
```

```
2 from scipy.interpolate import griddata
3
4 rng = np.random.default_rng(0)
5 points = rng.uniform([-3, -3], [3, 3], size=(200, 2))
6 values = np.sin(points[:, 0]) * np.cos(points[:, 1])
7
8 grid_x, grid_y = np.mgrid[-3:3:100j, -3:3:100j]
9 grid_z = griddata(points, values, (grid_x, grid_y), method='cubic')
10
11 # method can be 'nearest', 'linear', or 'cubic'
```

8.5 二维插值选型建议

- 规则网格：优先用 `RegularGridInterpolator`，速度快、结构清楚。
- 散点数据：数据点较少时可用 `griddata`；点很多时要考虑计算代价。
- 边界外：多数二维插值对凸包外外推不可靠，`griddata` 默认会给 `NaN`。
- 建模论文里要说明插值区域，不要把插值图解释成无限范围的真实规律。

9 方法对比与选型指南

9.1 按问题目标选方法

问题描述	优先方法	原因
函数表补值，节点少且准确	拉格朗日/牛顿插值	公式清楚，手算方便
节点不断增加	牛顿插值	新增节点只需增加一项差商
已知端点斜率或节点导数	Hermite 插值	可同时匹配函数值和导数
一维平滑曲线补值	三次样条	分段低次、整体光滑、稳定
数据含噪声但趋势平滑	最小二乘拟合/平滑样条	不强迫过噪声点
参数模型有物理含义	非线性最小二乘	能估计参数并解释模型
二维规则网格	RegularGridInterpolator	专门适配网格数据
二维散点	griddata/RBF	能处理非结构点

9.2 按考试题型选方法

- 1) 题目给 2–4 个点，要求“插值多项式”：优先写拉格朗日或牛顿。
- 2) 题目给差商表或要求新增节点：优先牛顿。
- 3) 题目给 $f(x_i)$ 和 $f'(x_i)$ ：Hermite。
- 4) 题目要求“自然三次样条”或“给边界条件”：列三对角方程。
- 5) 题目说“最佳平方逼近”：写内积、正交条件、解系数。
- 6) 题目说“最小二乘拟合直线/二次曲线”：列正规方程或设计矩阵。

9.3 按建模论文写法选方法

建模论文方法段模板

为获得连续函数估计，先对原始数据进行排序与异常值检查。由于数据（说明是否含噪声），本文采用（插值/拟合）方法。若为插值，构造函数 $s(x)$ 满足 $s(x_i) = y_i$ ；若为拟合，建立残差 $r_i = y_i - f(x_i; \theta)$ 并最小化 $\sum r_i^2$ 。随后通过残差图、RMSE、交叉验证或与已知数据对比评价模型效果。最终仅在观测区间内使用该模型，避免无依据外推。

10 完整 Python 代码模板

下面给出一个综合脚本，便于在建模作业中快速试验“插值”和“拟合”的区别。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.interpolate import BarycentricInterpolator, CubicSpline,
   PchipInterpolator
4 from scipy.optimize import curve_fit, least_squares
5
6 rng = np.random.default_rng(42)
7 x = np.linspace(0, 6, 12)
8 y_clean = 1.5 * np.exp(-0.45*x) + 0.25*x + 0.4
9 y = y_clean + rng.normal(0, 0.08, size=x.size)
10 xx = np.linspace(x.min(), x.max(), 400)
11
12 # 1. Polynomial least squares fit
13 coef = np.polyfit(x, y, deg=3)
14 y_poly = np.polyval(coef, xx)
15
16 # 2. Lagrange interpolation with barycentric representation
17 lag = BarycentricInterpolator(x, y)
18 y_lag = lag(xx)
19
20 # 3. Cubic spline interpolation
21 cs = CubicSpline(x, y, bc_type='natural')
22 y_cs = cs(xx)
23
24 # 4. Shape-preserving interpolation
25 pchip = PchipInterpolator(x, y)
26 y_pchip = pchip(xx)
27
28 # 5. Nonlinear least squares
29 def exp_model(t, a, b, c, d):
30     return a * np.exp(b*t) + c*t + d
31
32 popt, _ = curve_fit(exp_model, x, y, p0=[1.0, -0.5, 0.2, 0.0])
33 y_exp = exp_model(xx, *popt)
34
35 # Evaluation on the observed data
36 methods = {
37     'poly3': np.polyval(coef, x),
38     'lagrange': lag(x),
39     'spline': cs(x),
40     'pchip': pchip(x),
41     'nonlinear_ls': exp_model(x, *popt)
42 }
43 for name, pred in methods.items():
44     rmse = np.sqrt(np.mean((pred - y)**2))
45     print(name, 'RMSE=', rmse)
46
47 plt.scatter(x, y, label='data')
```

```
48 plt.plot(xx, y_poly, label='poly3 least squares')
49 plt.plot(xx, y_lag, label='Lagrange interpolation')
50 plt.plot(xx, y_cs, label='natural cubic spline')
51 plt.plot(xx, y_pchip, label='PCHIP')
52 plt.plot(xx, y_exp, label='nonlinear least squares')
53 plt.legend()
54 plt.show()
```


11 常见错误与改进策略

11.1 错误一：把含噪声数据强行插值

若数据来自实验测量，误差不可避免。强行插值会让曲线穿过每个噪声点，可能导致曲线抖动，预测变差。改进方法：使用最小二乘拟合、平滑样条、加权最小二乘或鲁棒损失。

11.2 错误二：多项式阶数越高越好

高阶多项式通常能降低训练误差，但可能出现端点振荡和外推爆炸。改进方法：降低阶数、使用 Chebyshev 节点、样条分段、正则化或交叉验证选阶。

11.3 错误三：直接求逆解最小二乘

正规方程 $A^T A \beta = A^T y$ 可以用于理论推导，但实际不应写成 $\beta = (A^T A)^{-1} A^T y$ 后直接计算。改进方法：用 `np.linalg.lstsq`、QR、SVD 或 SciPy 的线性代数接口。

11.4 错误四：忽略外推风险

插值和拟合通常只在数据覆盖区间内可靠。超出节点范围后，尤其是高次多项式、样条端点外延和非线性模型，都可能给出看似平滑但无意义的结果。

11.5 错误五：没有做残差诊断

最小二乘拟合后必须画残差图。如果残差呈趋势、周期或漏斗形，说明模型结构、方差假设或变量选择可能有问题。

12 复习结论

必须掌握的公式

- 1) 拉格朗日插值: $P_n(x) = \sum_{i=0}^n y_i \prod_{j \neq i} \frac{x-x_j}{x_i-x_j}$ 。
- 2) 拉格朗日误差: $f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x-x_i)$ 。
- 3) 牛顿插值: $P_n(x) = f[x_0] + f[x_0, x_1](x-x_0) + \cdots$ 。
- 4) 最小二乘: $\min \|A\beta - y\|_2^2$, 正规方程 $A^T A\beta = A^T y$ 。
- 5) 最佳平方逼近: 误差 $f - p$ 与逼近空间正交。
- 6) 三次样条: 分段三次、插值、 S' 连续、 S'' 连续, 再加边界条件。

建模时最重要的判断

- 数据是否含噪声? 含噪声优先拟合, 不含噪声才考虑严格插值。
- 是否需要导数连续? 需要光滑曲线优先三次样条或 Hermite。
- 是否是二维规则网格? 规则网格和散点数据用的工具不同。
- 是否要解释参数? 有物理含义的参数模型优先非线性最小二乘。
- 是否要预测? 一定要留出验证或做交叉验证。

13 参考资料

本讲义主要参考下列官方文档与论文资料, 方法介绍以数值分析常用教材表述为主, Python 接口以官方文档为准。

- [1] SciPy Documentation, *Interpolation (scipy.interpolate)*. <https://docs.scipy.org/doc/scipy/reference/interpolate.html>
- [2] SciPy User Guide, *Interpolation*. <https://docs.scipy.org/doc/scipy/tutorial/interpolate.html>
- [3] SciPy Documentation, *CubicSpline*. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.CubicSpline.html>
- [4] SciPy Documentation, *BarycentricInterpolator*. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.BarycentricInterpolator.html>
- [5] SciPy Documentation, *RegularGridInterpolator*. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.RegularGridInterpolator.html>
- [6] SciPy Documentation, *griddata*. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.griddata.html>

- [7] SciPy Documentation, *curve_fit*. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html
- [8] SciPy Documentation, *least_squares*. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.least_squares.html
- [9] NumPy Documentation, *numpy.linalg.lstsq*. <https://numpy.org/doc/stable/reference/generated/numpy.linalg.lstsq.html>
- [10] NumPy Documentation, *numpy.polynomial.polynomial.polyfit*. <https://numpy.org/doc/stable/reference/generated/numpy.polynomial.polynomial.polyfit.html>
- [11] Pauli Virtanen et al., *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*. <https://arxiv.org/abs/1907.10121>